



Instituto Nacional
de Astrofísica,
Óptica y Electrónica

Implementación de una aplicación de RNA: Identificación de canto de aves endémicas de Hawai

Inteligencia Computacional I

Edna Gabriela Gochicoa Fuentes (ednag.gochicoaf@inaoep.mx)

Daira Adriana Chavarría Rodríguez (daira.chavarriar@inaoep.mx)

28 de abril del 2026

Índice

Introducción.....	2
Conjunto de Datos.....	2
Implementación.....	2
Elección del enfoque.....	2
Preparación el conjunto de datos.....	2
División del conjunto de datos.....	3
Arquitectura de la red.....	5
Entrenamiento.....	5
Evaluación.....	6
Interfaz de Usuario.....	8
Experimentos.....	8
Prueba de Tolerancia al Ruido.....	8
Transferencia de Aprendizaje.....	10
Conclusiones.....	12
Referencias.....	12

Introducción

El problema abordado para esta tarea consistió en la clasificación automática de cantos de aves a partir de grabaciones de audio, utilizando técnicas de aprendizaje supervisado. A partir de un conjunto de grabaciones con anotaciones temporales, se extraen segmentos representativos de eventos correspondientes a diferentes especies. Estas señales son posteriormente transformadas al dominio tiempo-frecuencia mediante espectrogramas Log-Mel, los cuales nos permiten capturar de forma compacta la distribución espectral de la energía a lo largo del tiempo.

Sobre estas representaciones se implementa una Red Neuronal Artificial (RNA) de tipo convolucional (CNN), capaz de aprender patrones de espectrograma asociados a cada especie. El modelo es entrenado y evaluado usando métricas de desempeño y herramientas de análisis como gráficas y matriz de confusión, permitiendo evaluar su capacidad de generalización.

El objetivo, además, se centra en analizar el comportamiento del sistema bajo distintas condiciones experimentales. En particular, se busca identificar sus limitaciones e inducir situaciones en las que el sistema puede fallar.

Conjunto de Datos

Se utilizó el conjunto de datos “*A collection of fully-annotated soundscape recordings from the Island of Hawai’i*” disponible en Zenodo, el cual consiste en una colección de 635 grabaciones de audio con una duración total aproximada de 51 horas. Los datos fueron recolectados entre 2016 y 2022 en 4 sitios de la isla, por el Listening Observatory for Hawaiian Ecosystems (LOHE) de la Universidad de Hawai’i.

Implementación

Elección del enfoque

Para el desarrollo de este trabajo se consideraron dos estrategias de clasificación: el ajuste fino de un modelo especializado en audio (por ejemplo, YAMNet) y la implementación de una red neuronal convolucional (CNN) propia. El enfoque basado en YAMNet fue descartado tras obtener resultados insuficientes para los objetivos del proyecto. Por el contrario, la red entrenada desde cero demostró ser una solución viable para las etapas de entrenamiento y experimentación.

Preparación el conjunto de datos

A partir de las anotaciones temporales del conjunto de datos, se extrajeron segmentos de audio definidos por sus respectivos intervalos de inicio y fin, con el objetivo de asegurar que cada uno correspondiera a un evento acústico válido.

Posteriormente, estos segmentos fueron sometidos a un proceso de normalización de duración, mediante la aplicación de técnicas como acolchado o truncamiento, con el fin de garantizar consistencia en las dimensiones de entrada.

Una vez estandarizadas, las señales fueron transformadas al dominio tiempo-frecuencia mediante la generación de espectrogramas Log-Mel. Para evitar problemas de desbalance, se limitó el número máximo de muestras por clase.

La siguiente figura muestra la distribución de muestras por especie después del proceso de preparación del conjunto de datos. Como se puede observar, algunas clases alcanzan el límite máximo de 1000 muestras, mientras que otras presentan una cantidad significativamente menor, lo que evidencia el desbalance entre especies.

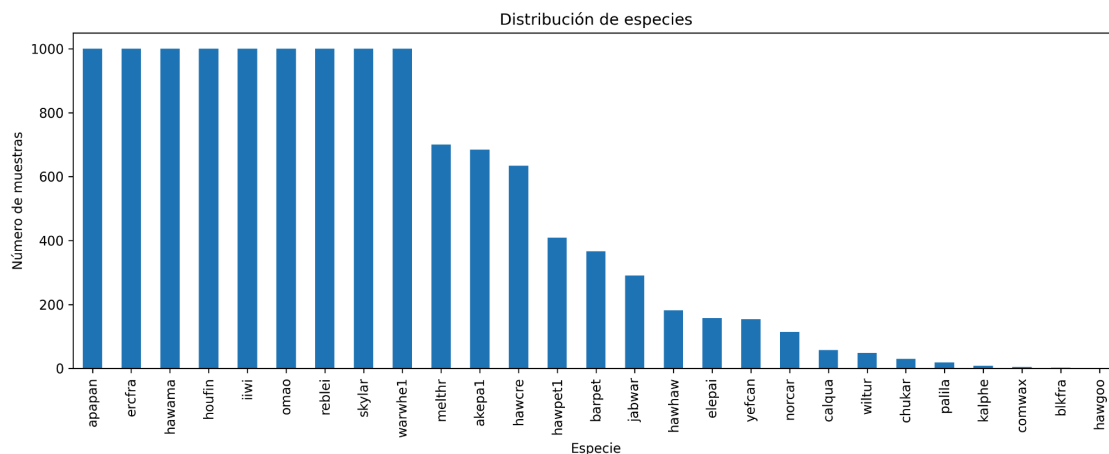


Figura 1. Distribución de muestras por especie después del proceso de preprocesamiento del conjunto de datos.

División del conjunto de datos

El conjunto de datos se dividió, mediante partición estratificada, en:

- Entrenamiento: 70
- Validación: 15
- Prueba: 15

Se separaron los datos en “clases principales” y “clases experimentales” con el fin de evaluar el comportamiento del modelo en escenarios experimentales específicos.

Para simplificar el entrenamiento de la red se aplicó una reducción en las clases mayoritarias y, como parte del proceso de balanceo, se descartaron aquellas clases con menos de 200 entradas ya que se consideraron tenían una cantidad insuficiente de datos. En total, la red fue entrenada con 7 especies de aves.

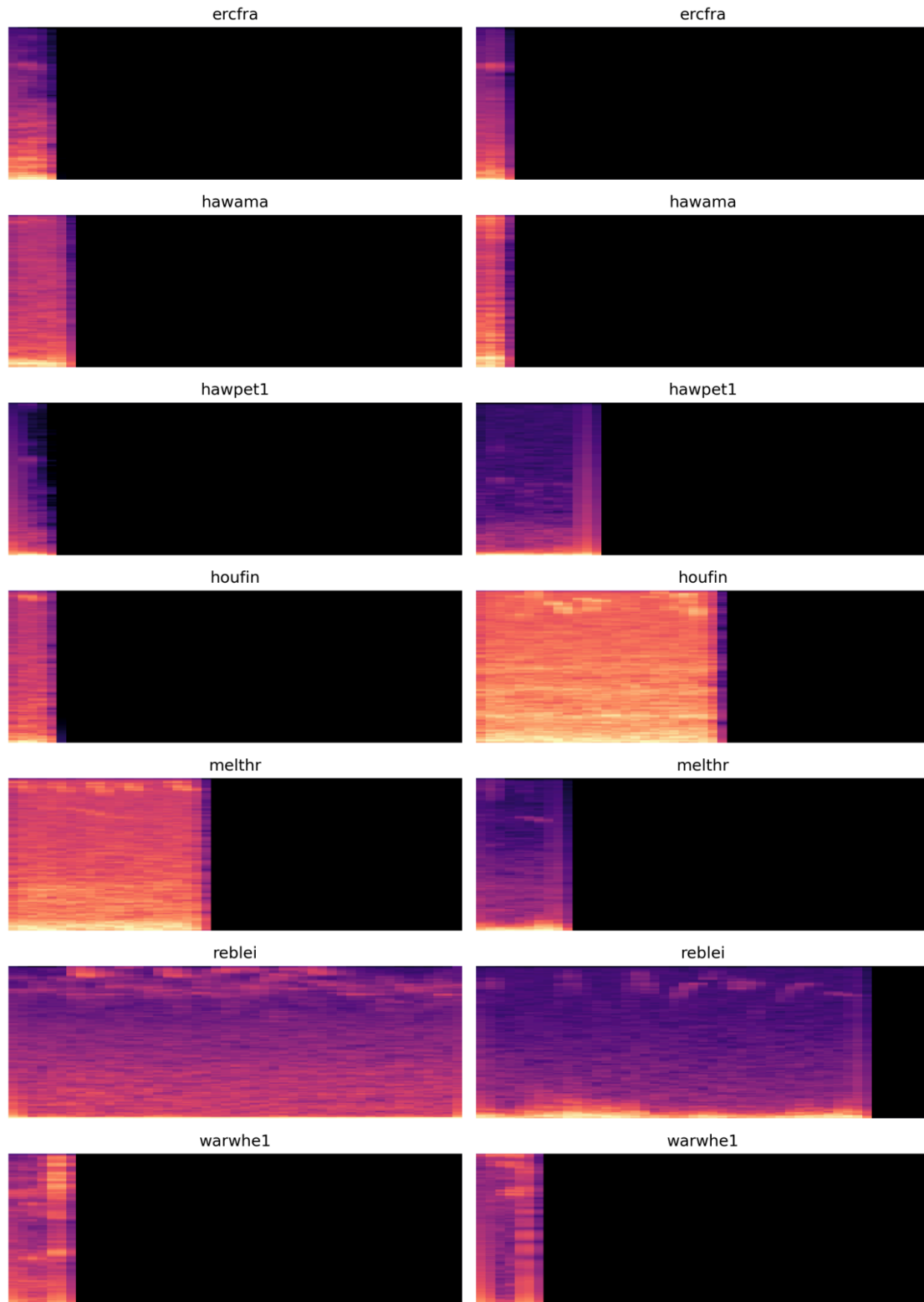


Figura 2. Ejemplos de espectrogramas de Log-Mel para las especies utilizadas en el entrenamiento

La figura presenta ejemplos de espectrogramas Log-Mel correspondientes a las clases utilizadas durante el entrenamiento del modelo. Cada fila representa una especie distinta, mientras que las columnas muestran diferentes instancias de la misma clase (variabilidad intra-clase).

Además de ello, se observa que algunas especies presentan patrones bien definidos y distribuidos a lo largo del tiempo (por ejemplo, “*rebeli*”), mientras que otras muestran actividad concentrada en intervalos temporales más cortos con una gran presencia de silencio durante el resto del segmento.

Arquitectura de la red

Se desarrolló una red neuronal convolucional orientada al procesamiento de espectrogramas Log-Mel, los cuales pueden interpretarse como representaciones bidimensionales de la señal de audio en el dominio tiempo-frecuencia.

Se usaron cuatro bloques convolucionales con la siguiente estructura:

- Capa Conv2D
- Capa de Batch Normalization
- Capa de MaxPooling2D

El número de filtros se incrementa progresivamente a lo largo de la red, lo cual permite que el modelo capture características cada vez más complejas. En las primeras capas se aprenden patrones simples relacionados con componentes espectrales básicos, mientras que en capas más profundas se identifican estructuras más abstractas.

“Batch Normalization” en cada bloque contribuye a estabilizar el entrenamiento. Por su parte, las capas de “Max Pooling” reducen la dimensionalidad espacial de las representaciones, disminuyendo a su vez el costo computacional y ayudando a la red a enfocarse en las características más relevantes.

Posteriormente, se emplea una capa de “Global Average Pooling”, la cual sustituye a las capas tradicionales de “*flattening*”. Esta operación reduce cada mapa de características a un único valor promedio, disminuyendo significativamente el número de parámetros del modelo y reduciendo el riesgo de sobreajuste.

Adicionalmente, se incorporan capas densas completamente conectadas. Se utiliza una capa intermedia con 256 neuronas y activación ReLU, seguida de una capa de “dropout”, desactivación aleatoria de neuronas, con probabilidad de 0.4. Finalmente se emplea una función de activación “softmax” para la capa de salida, generando una distribución de probabilidad sobre las clases de aves.

Entrenamiento

El modelo fue entrenado utilizando el optimizador Adam con una tasa de aprendizaje de 1×10^{-4} , favoreciendo una convergencia estable. Además, se utilizó la función de pérdida de entropía cruzada categórica dispersa.

Para trabajar el desbalance en el conjunto de datos, se calcularon pesos de clase que penalizan de forma diferenciada los errores según la frecuencia de cada clase. Adicionalmente, se aplicó “*early stopping*” para detener el entrenamiento cuando el desempeño en validación deja de mejorar, y “*model checkpointing*” para conservar la mejor versión del modelo.

Evaluación

El desempeño del modelo se evaluó usando métricas de precisión, exactitud, sensibilidad y F1 por clase.

Las curvas de pérdida muestran una disminución progresiva tanto en entrenamiento como en validación. Sin embargo, se observa que la pérdida de entrenamiento continúa decreciendo de manera sostenida, mientras que la pérdida de validación presenta fluctuaciones y tiende a estabilizarse alrededor de la época 30.

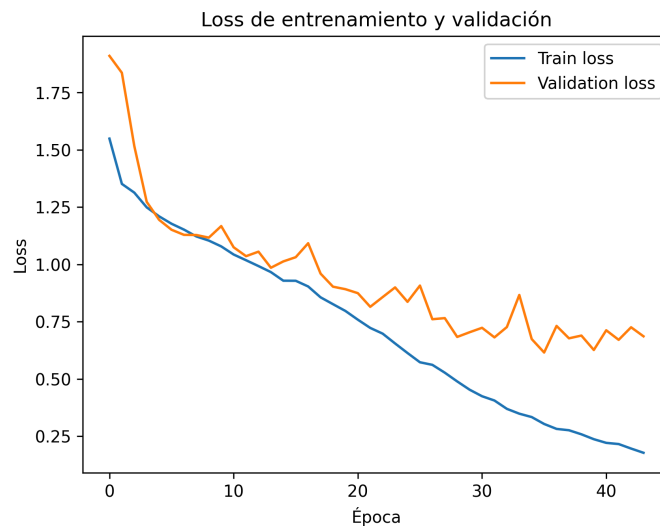


Figura 3. Curva de pérdida (loss) durante el entrenamiento y validación del modelo a lo largo de las épocas.

Las curvas obtenidas en precisión reflejan que el modelo alcanza un valor cercano al 90% en entrenamiento, mientras que en validación se estabiliza alrededor de 75-80%. La brecha entre estas curvas confirma la existencia de una diferencia en la capacidad de generalización del modelo, lo cual es esperable en presencia de ruido, variabilidad en las señales o desbalance entre clases.

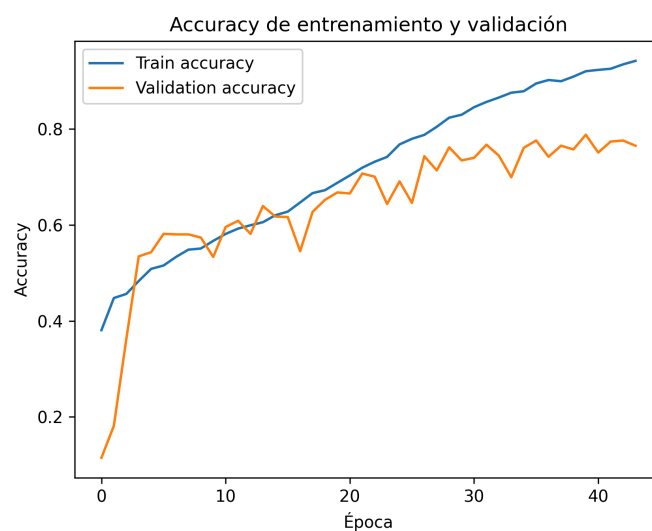


Figura 4. Curva de precisión para los conjuntos de entrenamiento y validación.

A través de la matriz de confusión, observamos un buen desempeño en clases como: “ercfra”, “rebeli” y “warwhe1”. Por otro lado, existen confusiones considerables entre ciertas clases, como “houfin” y “hawama”, lo que sugiere la existencia de una gran similitud en sus patrones acústicos, o insuficiente discriminación en las características extraídas.

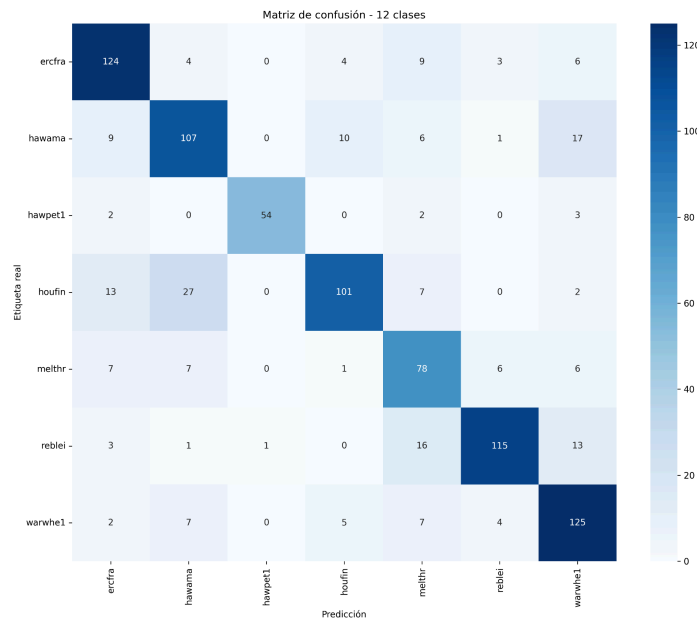


Figura 5. Matriz de confusión del modelo en el conjunto de prueba.

Interfaz de Usuario

Se desarrolló una interfaz utilizando la librería “Gradio”.

La aplicación se diseñó con una estructura modular que divide su funcionalidad en tres modos: evaluación sobre el conjunto de prueba, clasificación de audio externo y evaluación de un clasificador que utiliza el clasificador como base con una capa de ajuste fino.

- Para el primer caso, el usuario puede seleccionar un índice correspondiente a una muestra en el conjunto de datos e incrementar el ruido de la misma. Después de evaluarla, se puede observar la predicción del modelo junto con la etiqueta real, así como un top 5 de candidatos de predicción y su grado de confianza para cada uno.
- En el segundo caso, se permite al usuario cargar archivos de audio propios (ya sean correspondientes al conjunto de datos o externos), lo que posibilita analizar el comportamiento del modelo ante señales no vistas durante el entrenamiento, por ejemplo. Al igual que el primer caso, se muestra un top 5 de candidatos y su grado de confianza en predicción.
- El tercer caso, permite al usuario seleccionar una muestra de un conjunto de datos diferente y observar la predicción de un modelo entrenado a partir del modelo principal.

Se presenta la información en un formato organizado por columnas, donde se muestran de forma simultánea: la predicción del modelo, las probabilidades asociadas a las clases más relevantes, el espectrograma de la señal analizada, y una imagen representativa de la especie detectada.

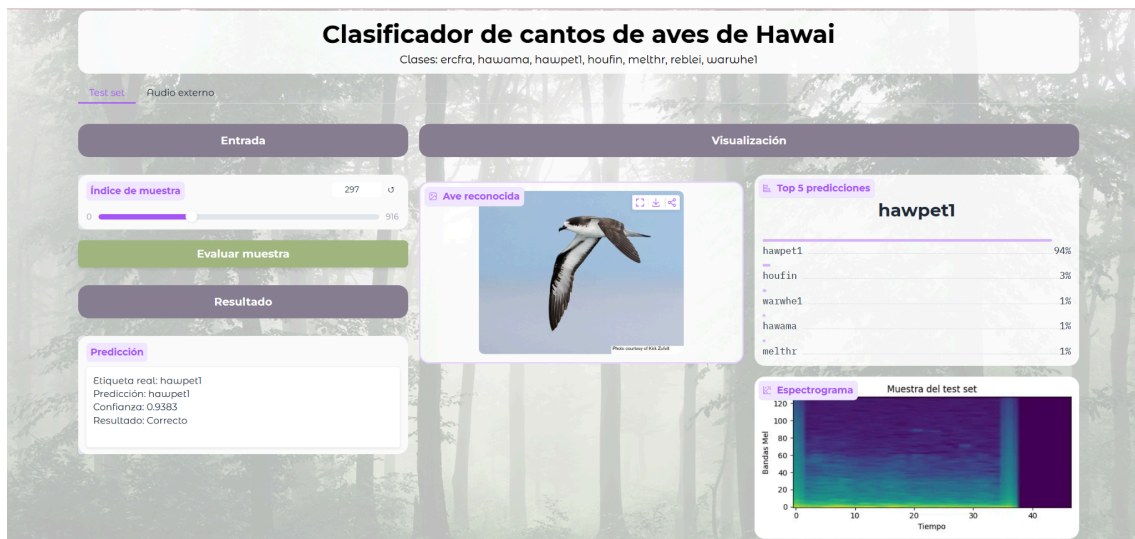


Figura 6. Funcionamiento de la interfaz de usuario desarrollada.

Experimentos

Prueba de Tolerancia al Ruido

Para evaluar la robustez del modelo se analizó el desempeño del clasificador ante señales ruidosas. La intensidad del ruido se controló mediante la relación señal-ruido (SNR); se observó una degradación significativa del desempeño cuando la SNR cayó por debajo de 45 dB.

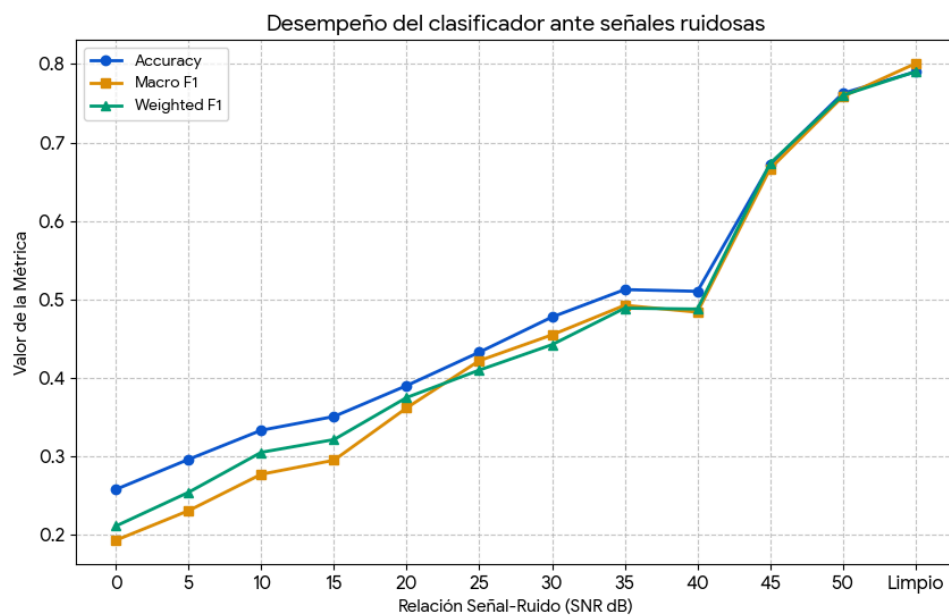


Figura 7. Evaluación de la robustez del clasificador frente a ruido temporal.

Se presentó al usuario una versión simplificada del experimento, en la que puede modificar la señal de audio agregando ruido blanco y observar cómo cambia la clasificación.

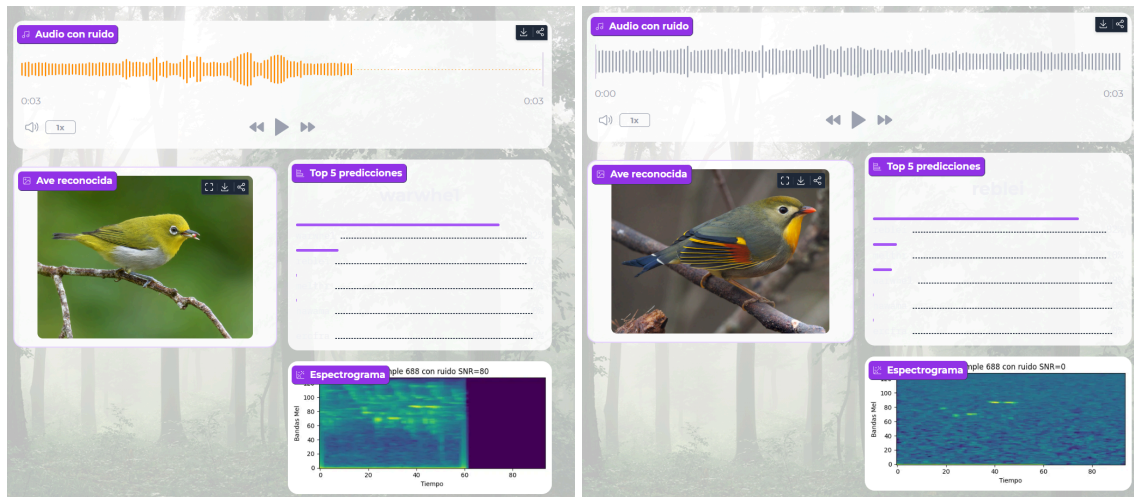


Figura 8. Comparación de la señal original y la señal con un SNR de 0. El modelo clasifica equivocadamente con alta confianza

Clasificación de datos fuera de dominio

Usando ruido blanco aleatorio y las clases del conjunto de experimentación, compuesto por aves no incluidas en el entrenamiento de la CNN, se evaluó la robustez del clasificador ante datos fuera de dominio.

Al emplear sonidos de aves *sin relación* con las clases del entrenamiento, el modelo mantuvo una distribución similar a la del conjunto original. Aunque la confianza promedio fue ligeramente menor que con datos del dominio, siguió siendo alta, lo que evidencia sobreconfianza y falta de generalización frente a entidades no pertenecientes al dominio de la clasificación.

Ante ruido, el modelo tiende a clasificar la mayoría de los casos como “*ercfra*”, lo que indica sesgo hacia esta clase o una mayor sensibilidad de sus características al ruido. Esto puede significar que el modelo está sobreajustado o es poco robusto frente a ruido aleatorio.

Tabla 3. Confianza del modelo en datos sin relación, ruido aleatorio y set de validación

Métrica	Ruido Aleatorio	Datos OOD	Datos Conocidos
Promedio	0.7714	0.7767	0.8322
Mediana	0.7891	0.8308	0.9189
Desviación Estándar	—	0.1930	0.1881
Mínimo	0.3407	0.2772	0.2566
Máximo	0.9793	1.0000	1.0000

Para la demostración al usuario se decidió agregar un módulo donde se le permita introducir un audio de su elección y observar la confianza de predicción del modelo.

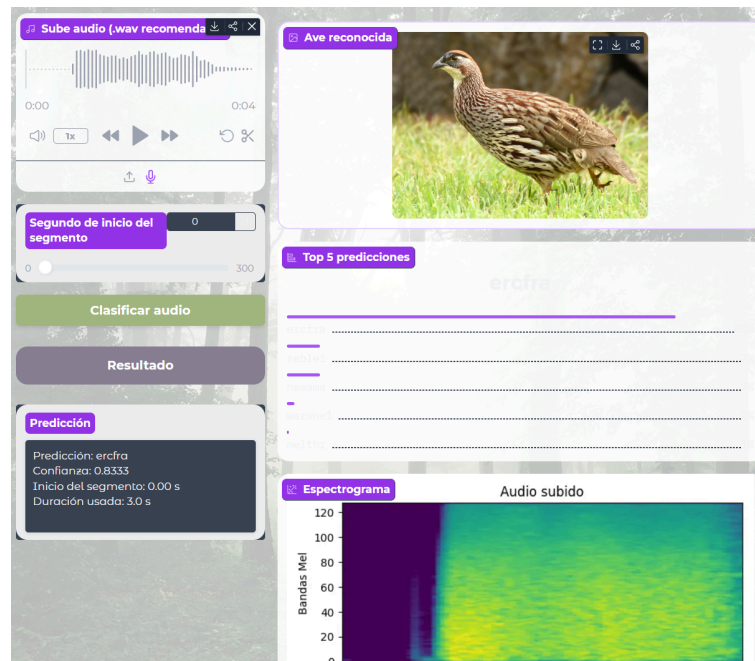


Figura 10. Desempeño de la aplicación con datos fuera del dominio.

Transferencia de Aprendizaje

Se evaluó la capacidad del modelo para adaptarse a nuevas clases a través de un proceso de “*ajuste fino*” en el cual se reutilizaron las capas convolucionales previamente entrenadas, reemplazando únicamente la de salida por una nueva, correspondiente a clases no vistas.

Durante el experimento, las capas base fueron congeladas, permitiendo que únicamente la capa final pudiera aprender a mapear las nuevas representaciones a las clases correspondientes.

Tabla 5. Aprendizaje por transferencia a otras especies de Aves

Clase	Precisión	Recall	F1-score	Soporte
akepal	0.39	0.27	0.32	102
apapan	0.53	0.53	0.53	150
barpet	0.95	0.98	0.96	55
hawcre	0.69	0.23	0.35	95
iiwi	0.35	0.43	0.39	150
jabwar	0.75	0.70	0.72	43
omao	0.54	0.66	0.59	150
skylar	0.72	0.86	0.78	149
Accuracy		0.56		894
Macro avg	0.61	0.58	0.58	894
Weighted avg	0.57	0.56	0.55	894

Los resultados muestran que el modelo es capaz de adaptarse parcialmente a nuevas especies, no incluidas en el entrenamiento. Como se puede observar en la tabla 5, el desempeño global alcanza una precisión de 0.56, con valores de Macro F1 de 0.58 y de F1 Ponderado de 0.55.

A nivel de clase, se puede observar una variabilidad significativa en el desempeño. Por ejemplo, “*barpet*” presenta métricas elevadas (F1-score = 0.96), mientras que “*akepa*” muestra valores más bajos, especialmente en sensibilidad (0.27). A través de ello se infiere que, a pesar de que las capas convolucionales hayan aprendido representaciones generales de utilidad, la separabilidad entre clases depende bastante de la similitud entre las nuevas especies y aquellas utilizadas durante el entrenamiento. Clases con patrones acústicos menos definidos, o más cercanos a otras especies, tienden a presentar más errores.

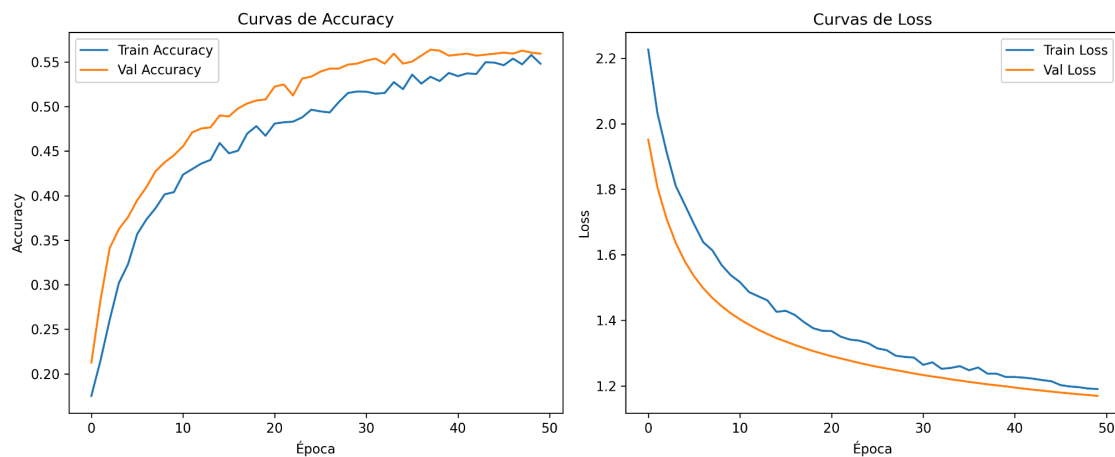


Figura 10. Curvas de precisión y pérdida para el experimento de ajuste fino

Las curvas de entrenamiento muestran una convergencia estable tanto en *precisión* como en *pérdida*. Asimismo, la curva de validación se mantiene por encima de la de entrenamiento en precisión (no hay sobreajuste significativo). La disminución progresiva del *pérdida* sugiere que el modelo aprende de forma consistente durante el proceso de *ajuste fino*. Sin embargo, las características aprendidas no son suficientemente óptimas para discriminar nuevas clases sin un ajuste más profundo de la red.



Figura 11. Resultados de ajuste fino a través de la interfaz de usuario desarrollada.

Conclusiones

A partir de los experimentos realizados, observamos que:

- El modelo logró aprender representaciones útiles a partir de espectrogramas Log-Mel, logrando un desempeño adecuado cuando las señales de entrada contienen patrones claros y consistentes, debido a la efectividad de las CNN para el análisis de señales tiempo-frecuencia.
- Se evidenció que el modelo presenta una alta dependencia de la calidad de la señal acústica, debido a la degradación del desempeño al ir introduciendo ruido. Es decir, no generaliza completamente ante variaciones externas,
- Al evaluar el modelo con ruido aleatorio y datos no relacionados, el modelo asigna clases conocidas incluso a entradas con únicamente ruido blanco. Eso indica que el modelo siempre fuerza una predicción, aunque el resultado sea incorrecto (consistente con los modelos de clasificación supervisada).
- En relación a Transfer Learning, no se esperaba que el modelo pudiera adaptarse a nuevas clases sin reentrenar la red. Sin embargo, los resultados muestran que, incluso al congelar las capas base y ajustar únicamente la capa final, el modelo logra un desempeño simbólico/moderado.

Además de ello, la variabilidad de su desempeño por clase indica que la capacidad de transferencia no es uniforme. Es decir, las clases con características más distintivas se benefician más de las representaciones previas, a comparación de aquellas con mayor similitud entre sí, o mayor variabilidad.

Referencias

1. Navine, A., Kahl, S., Tanimoto-Johnson, A., Klinck, H., & Hart, P. (2022). A collection of fully-annotated soundscape recordings from the Island of Hawai'i [Conjunto de datos]. En Zenodo (CERN European Organization for Nuclear Research). <https://doi.org/10.5281/zenodo.7078499>
2. Tensorflow. (s. f.). *GitHub - tensorflow/models: Models and examples built with TensorFlow*. GitHub. <https://github.com/tensorflow/models>
3. *Tf.keras.layers.MelSpectrogram* | *TensorFlow v2.16.1*. (s. f.). TensorFlow. https://www.tensorflow.org/api_docs/python/tf/keras/layers/MelSpectrogram
4. Bex Tuychiev. (2024, July 29). *Construir interfaces de usuario para aplicaciones de IA con Gradio en Python*. Datacamp.com; DataCamp. <https://www.datacamp.com/es/tutorial/gradio-python-tutorial>
5. Gradio-App. (s. f.). *GitHub - gradio-app/gradio: Build and share delightful machine learning apps, all in Python*. Start to support our work! GitHub. <https://github.com/gradio-app/gradio>
6. Jesús Utrera Burgal. (2018, July 11). Deep Learning básico con Keras (Parte 2): Convolutional Nets. En Mi Local Funciona. <https://www.enmilocalfunciona.io/deep-learning-basico-con-keras-parte-2-convolutional-nets/>